DBMS TASK 8

SQL Retrieval and Summary Calculations

Purpose

Demonstrate common SQL retrieval clauses and aggregate functions using the sample records in OnlineShoppingDB.

A. Filter by category — WHERE

```
SELECT ProductID, ProductName, Price
FROM Products
WHERE Category = 'Accessories';
```

What it shows: Returns only products classified as accessories.

B. Arrange products by price — ORDER BY

```
SELECT ProductName, Price
FROM Products
ORDER BY Price DESC;
```

What it shows: Lists the most expensive products first.

C. Count products in each category — GROUP BY

```
SELECT Category, COUNT(*) AS ProductCount
FROM Products
GROUP BY Category;
```

What it shows: Groups product rows by category and counts the items in each group.

D. Keep categories with several products — HAVING

```
SELECT Category, COUNT(*) AS ProductCount
FROM Products
GROUP BY Category
HAVING COUNT(*) > 1;
```

What it shows: Filters groups after aggregation. The sample data should show Accessories and Electronics.

E. Number of orders — COUNT()

```
SELECT COUNT(*) AS TotalOrders
FROM Orders;
```

What it shows: Counts all order headers; the sample data contains six.

F. Overall order-line value — SUM()

```
SELECT SUM(Quantity * UnitPrice) AS TotalOrderValue
FROM OrderDetails;
```

What it shows: Adds each order line's quantity multiplied by its recorded unit price; the sample total is 113100.00.

G. Mean catalogue price — AVG()

```
SELECT ROUND(AVG(Price), 2) AS AverageProductPrice
FROM Products;
```

What it shows: Calculates the mean price of the five products; the sample result is 17360.00.

H. Cheapest item — MIN()

```
SELECT MIN(Price) AS LowestPrice
FROM Products;
```

What it shows: Returns the lowest catalogue price, which is 800.00 in the sample.

I. Most expensive item — MAX()

```
SELECT MAX(Price) AS HighestPrice
FROM Products;
```

What it shows: Returns the largest catalogue price, which is 55000.00 in the sample.

J. Orders after a selected date — WHERE with ORDER BY

```
SELECT OrderID, CustomerID, OrderDate, Status
FROM Orders
WHERE OrderDate >= '2026-10-04'
ORDER BY OrderDate;
```

What it shows: Filters orders by date and displays the matching orders in date order.

Reference Values from the Sample Records

Calculation	Sample result
COUNT(Orders)	6
SUM(Quantity × UnitPrice)	113,100.00
AVG(Products.Price)	17,360.00
MIN(Products.Price)	800.00
MAX(Products.Price)	55,000.00

Run the statements in Workbench and add screenshots of your own output grids. Values may change if your dataset differs.

Closing Remarks

The examples show how filtering, sorting, grouping, and aggregation turn table data into concise information useful for checking orders and product prices.

SQL CODE — MYSQL WORKBENCH

Run each SELECT separately and capture its result grid.

```
USE OnlineShoppingDB;

-- WHERE
SELECT ProductID, ProductName, Price FROM Products
WHERE Category = 'Accessories';

-- ORDER BY
SELECT ProductName, Price FROM Products ORDER BY Price DESC;

-- GROUP BY
SELECT Category, COUNT(*) AS ProductCount
FROM Products GROUP BY Category;

-- HAVING
SELECT Category, COUNT(*) AS ProductCount
FROM Products GROUP BY Category
HAVING COUNT(*) > 1;

-- COUNT
SELECT COUNT(*) AS TotalOrders FROM Orders;

-- SUM
SELECT SUM(Quantity * UnitPrice) AS TotalOrderValue
FROM OrderDetails;

-- AVG
SELECT ROUND(AVG(Price), 2) AS AverageProductPrice FROM Products;

-- MIN and MAX
SELECT MIN(Price) AS LowestPrice FROM Products;
SELECT MAX(Price) AS HighestPrice FROM Products;

-- WHERE with ORDER BY
SELECT OrderID, CustomerID, OrderDate, Status
FROM Orders WHERE OrderDate >= '2026-10-04'
ORDER BY OrderDate;
```